A Project Report On

Click Fraud Detection Using Machine Learning

Submitted in partial fulfillment of the requirement for the award
of the degree

Master of Computer Applications (MCA)

from

Marwadi University

Academic Year 2026 – 27

Kruti Joshi – [92500584081]

Vaidik Pandya – [92500584114]

Internal Guide

Dr. Divyakant Meva

Rajkot-Morbi Road, At & PO: Gauridad, Rajkot 360 003, Gujarat, India.

Faculty of Computer Applications (FoCA)

Certificate

This is to certify that the project work entitled

Click Fraud Detection Using Machine Learning

submitted in partial fulfillment of the requirement for the award of the degree of

Master of Computer Applications (MCA)

of

Marwadi University

is a result of the bonafide work carried out by

Kruti Joshi – [92500584081]

Vaidik Pandya – [92500584114]

during the academic year 2026 – 2027

Faculty Guide	HOD	Dean

DECLARATION

I hereby declare that the project work entitled "Click Fraud Detection Using Machine Learning" is an original record of the work carried out by me as part of my academic project.

The analysis, implementation, observations and documentation presented in this report are based on the submitted project files and dataset. Wherever standard machine learning concepts are discussed, they have been explained in my own words for academic understanding. This work has not been submitted to any other university or institute for the fulfillment of another course or degree.

Place: Rajkot, Gujarat

Date: _____

Kruti Joshi – [92500584081] Signature: _____

Vaidik Pandya – [92500584114] Signature: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during the development of this project. Completing a machine learning project requires more than writing code; it also involves understanding the problem, preparing the data carefully, testing different approaches and learning from results that do not always behave as expected.

I am especially thankful to my project guide, Dr. Divyakant Meva, for the guidance, suggestions and encouragement provided throughout the work. The feedback received during the implementation and documentation stages helped me improve both the technical approach and the way the project is presented.

I also thank the faculty members of Marwadi University for providing the academic environment and resources needed for the project. Finally, I am grateful to my friends and classmates for their discussions, motivation and practical support during testing and documentation.

Kruti Joshi – [92500584081] Signature: _____

Vaidik Pandya – [92500584114] Signature: _____

CONTENTS

FIGURE INDEX

TABLE INDEX

1. Introduction

Online advertising depends heavily on click activity. A genuine click normally represents a real user showing interest in an advertisement, while a fraudulent click may be produced by bots, automated scripts, repeated artificial interactions or suspicious traffic sources. When fraudulent clicks are mixed with genuine traffic, advertising budgets can be wasted and campaign analytics can become misleading.

The aim of this project is to build a machine learning based system that classifies each click as fraudulent or non-fraudulent. Instead of relying on one rule, the project studies several signals at the same time: user interaction behaviour, click timing, device and browser information, network-related indicators, URL properties and a bot-likelihood score. The target variable used for training is is_fraudulent.

Five supervised classification algorithms are implemented: Logistic Regression, Gaussian Naive Bayes, K-Nearest Neighbors (KNN), Decision Tree and Random Forest. Each model is placed inside a preprocessing pipeline. Hyperparameters are searched with GridSearchCV using stratified five-fold cross-validation and F1 score as the optimization criterion. This makes the workflow systematic and allows different model families to be compared under a common evaluation process.

1.1 Project Objectives

- Study the characteristics of the click-fraud dataset and identify the most useful behavioural and technical fields.
- Create a reusable preprocessing pipeline for mixed numerical and categorical data.
- Engineer meaningful features from timestamps and URL strings while removing high-cardinality identifiers.
- Train and compare five classification algorithms using a consistent train-test split and cross-validation strategy.
- Evaluate the models using accuracy, precision, recall, F1 score, confusion matrix and classification report.
- Understand the effect of class imbalance and identify practical improvements for fraud detection.

2. Dataset Description

2.1 Overview

Property	Details
Dataset File	click_fraud_dataset_cleaned.csv
Total Records	55,000
Total Columns	21
Target Variable	is_fraudulent
Non-Fraudulent Records	41,172 (74.86%)
Fraudulent Records	13,828 (25.14%)
Missing Values	0
Duplicate Rows	0
Task Type	Supervised Binary Classification

Table 1: Dataset Overview

The dataset contains 55,000 click records with 21 columns. The target is not perfectly balanced: approximately three quarters of the observations belong to the non-fraudulent class, while roughly one quarter are labelled fraudulent. This imbalance is important because a model can achieve deceptively high accuracy by predicting the majority class too often.

2.2 Nature of the Input Data

The fields represent several aspects of a click event. Some are direct behavioural measurements such as click duration, scroll depth, mouse movement and keystrokes. Others describe the environment of the click, including device type, browser, operating system, ad position, VPN or proxy use and IP reputation. Timestamp and URL columns are used for feature engineering rather than being treated as raw strings.

2.3 Feature Matrix

Column	Description	Type
click_id	Unique click identifier	Identifier
timestamp	Date and time of the click	Datetime
user_id	Unique user identifier	Identifier
ip_address	Source IP address	Identifier
device_type	Desktop / mobile / tablet etc.	Categorical
browser	Browser used for the click	Categorical
operating_system	Operating system	Categorical
referrer_url	Page that referred the user	Text / URL
page_url	Destination page	Text / URL

Column	Description	Type
click_duration	Duration of the click/session interaction	Numeric
scroll_depth	Amount of page scrolling	Numeric
mouse_movement	Recorded mouse movement	Numeric
keystrokes_detected	Count of detected keystrokes	Numeric
ad_position	Placement of advertisement	Categorical
click_frequency	Number/frequency of clicks	Numeric
time_since_last_click	Time gap from previous click	Numeric
device_ip_reputation	Reputation category of device/IP	Categorical
VPN_usage	VPN usage flag	Binary
proxy_usage	Proxy usage flag	Binary
bot_likelihood_score	Estimated likelihood of bot activity	Numeric
is_fraudulent	Classification label: 0 or 1	Target

Table 2: Feature Matrix — All Dataset Columns

2.4 Descriptive Statistics

Feature	Mean	Std. Dev.	Min	Max
click_duration	1.83	1.82	0.00	18.28
scroll_depth	49.65	28.93	0.00	99.00
mouse_movement	250.36	144.18	0.00	499.00
keystrokes_detected	24.64	14.42	0.00	49.00
click_frequency	5.00	2.58	1.00	9.00
time_since_last_click	300.09	172.18	1.00	599.00
bot_likelihood_score	0.50	0.29	0.00	1.00

Table 3: Selected Descriptive Statistics

The numerical fields are measured on very different scales. For example, bot_likelihood_score is bounded on a small decimal scale, whereas mouse movement and time-based fields can be much larger. This difference justifies standardization before distance-based or linear models are trained.

2.5 Problem Characteristics

Click-fraud detection is not only a classification problem; it is also an imbalanced classification problem. Missing a fraudulent click can be costly, but incorrectly flagging a genuine click is also undesirable. For that reason, F1 score is used as the main GridSearchCV scoring metric because it balances precision and recall rather than focusing on overall accuracy alone.

3. Data Preprocessing & Exploratory Analysis

3.1 Feature Engineering and Cleaning

The preprocessing module first loads the CSV and checks the number of missing values. It then creates additional features before dropping columns that behave mainly as identifiers. The timestamp is converted into hour, day, month and day_of_week. Referrer and page URLs are transformed into simple structural features such as string length and whether the address starts with HTTPS.

Step	Implementation
Timestamp features	hour, day, month, day_of_week
Referrer features	referrer_length, referrer_https
Page URL features	page_url_length, page_https
Removed identifiers	click_id, user_id, ip_address, timestamp, referrer_url, page_url
Numerical missing values	Median imputation
Numerical scaling	StandardScaler
Categorical missing values	Most-frequent imputation
Categorical encoding	OneHotEncoder(handle_unknown="ignore")
Dimensionality reduction	PCA; candidate retained variance 90% or 95%

Table 4: Preprocessing Strategy

3.2 Train-Test Split

All model scripts use train_test_split with test_size=0.20, random_state=42 and stratify=y. As a result, about 44,000 records are used for training and 11,000 records for testing while preserving the target-class ratio in both subsets.

3.3 Exploratory Data Analysis

The exploratory analysis script checks dataset shape, column names, statistical summary, missing values, duplicate rows and the target distribution. It also produces boxplots for important behavioural features, scatter plots for selected variable pairs and a correlation heatmap for numerical fields.

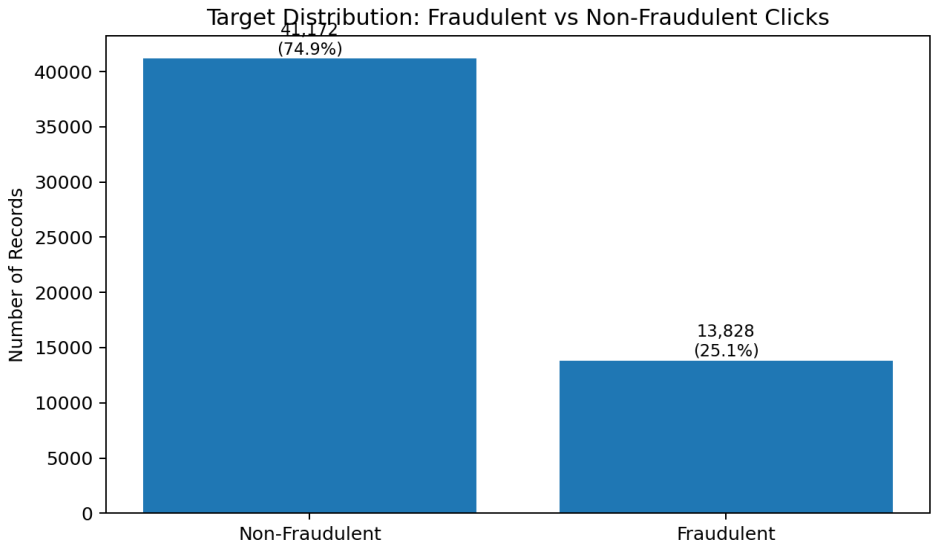

Figure 1: Target Distribution

The class distribution is the most important early observation. The dataset contains 41,172 non-fraudulent clicks and 13,828 fraudulent clicks. A classifier that predicts only class 0 would therefore obtain about 74.86% accuracy, but it would have zero recall for fraud. This becomes a useful baseline when interpreting the model results.

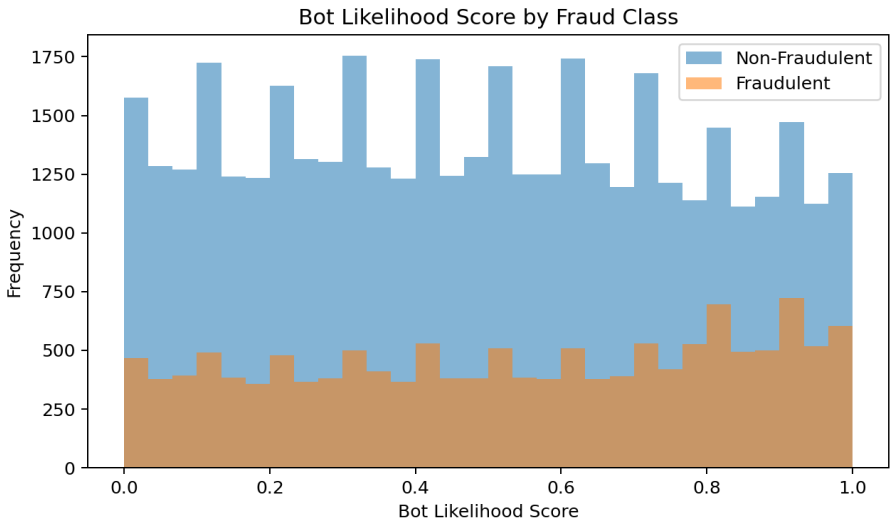

Figure 2: Bot Likelihood Score by Fraud Class

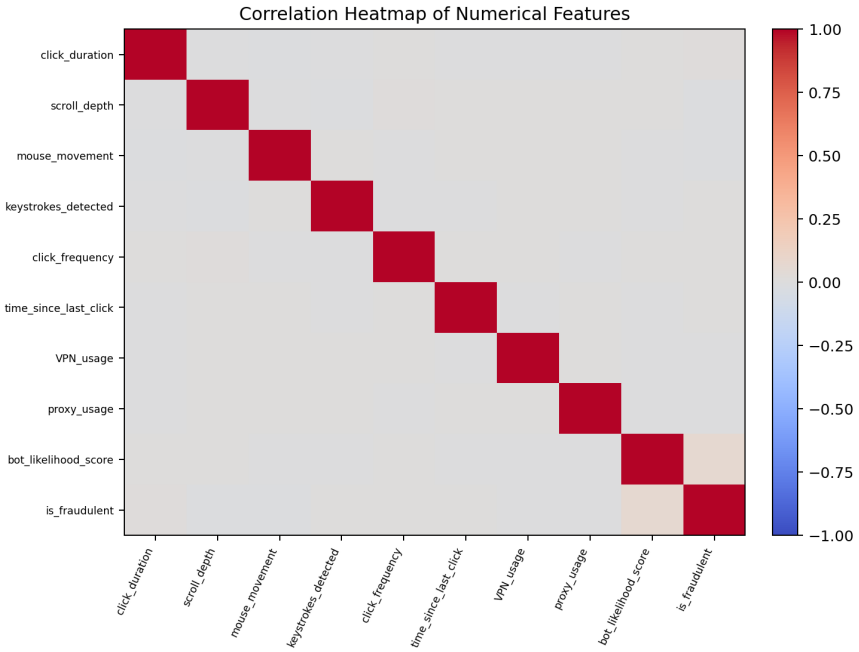

Figure 3: Correlation Heatmap

The visual analysis is useful for understanding overlap between classes. No single plotted feature should be treated as a complete fraud rule; the purpose of machine learning here is to combine multiple weak or moderate signals into a classification decision.

4. Machine Learning Models Used

The project implements five classifiers. Every model uses the same preprocessing function and a stratified 80/20 split. A Pipeline combines preprocessing, PCA and the classifier so that transformation steps are learned from training data inside the model workflow.

4.1 Logistic Regression

Logistic Regression is a linear probabilistic classifier. It estimates the probability that a record belongs to the fraudulent class by applying a logistic function to a weighted combination of the input features. In this project, max_iter is increased to 2000 so the optimization has enough iterations to converge.

- PCA retained variance candidates: 0.90 and 0.95.
- Regularization strength C candidates: 0.1, 1 and 10.
- Five-fold StratifiedKFold cross-validation.
- GridSearchCV scoring metric: F1 score.

The linear model is useful as a baseline, but it can struggle when the distinction between genuine and fraudulent behaviour is non-linear or when the decision threshold is not suitable for an imbalanced target.

4.2 Gaussian Naive Bayes

Gaussian Naive Bayes models each class using conditional probability and assumes that the transformed numerical components follow Gaussian-like distributions. It is computationally light and often works well as a quick baseline, although its independence assumption can be restrictive when features are strongly related.

- PCA retained variance candidates: 0.90 and 0.95.
- var_smoothing candidates: 1e-9, 1e-8 and 1e-7.
- Five-fold stratified cross-validation with F1 optimization.

4.3 K-Nearest Neighbors

KNN classifies a new sample by looking at the labels of nearby samples in the transformed feature space. Because the method relies on distance, scaling and PCA are especially important. The submitted implementation tests both uniform and distance-based voting.

- n_neighbors candidates: 3, 5 and 7.
- weights candidates: uniform and distance.
- PCA candidates: 90% and 95% retained variance.

KNN can capture local patterns without assuming a fixed equation, but prediction cost increases with dataset size and performance can fall when classes overlap in the feature space.

4.4 Decision Tree

A Decision Tree creates a sequence of if-then splits. Each split attempts to create purer groups with respect to the fraud label. This makes the model easy to interpret conceptually and able to learn non-linear relationships.

- criterion candidates: gini and entropy.
- max_depth candidates: 5, 10 and 20.
- random_state=42 for reproducibility.
- PCA retained variance candidates: 0.90 and 0.95.

Limiting maximum depth is important because an unrestricted tree can memorize training patterns and lose generalization on unseen clicks.

4.5 Random Forest

Random Forest combines many decision trees. Each tree sees a bootstrap sample of the training data and a random subset of features, and the final class is based on the ensemble vote. The method usually reduces the variance of a single decision tree and can represent complex feature interactions.

- n_estimators candidates: 100 and 200.
- max_depth candidates: 10 and 20.
- n_jobs=-1 to use available CPU cores.
- Five-fold stratified cross-validation with F1 scoring.

4.6 Model Configuration Summary

Model	Key Parameters Searched	CV / Score
Logistic Regression	C: 0.1, 1, 10; PCA: 0.90, 0.95	5-fold / F1
Gaussian Naive Bayes	var_smoothing: 1e-9, 1e-8, 1e-7; PCA: 0.90, 0.95	5-fold / F1
KNN	k: 3, 5, 7; weights: uniform/distance; PCA: 0.90, 0.95	5-fold / F1
Decision Tree	criterion: gini/entropy; depth: 5, 10, 20; PCA: 0.90, 0.95	5-fold / F1
Random Forest	trees: 100/200; depth: 10/20; PCA: 0.90, 0.95	5-fold / F1

Table 5: Model Configuration

Using GridSearchCV makes the comparison more disciplined, but it does not automatically solve class imbalance. If every parameter combination prefers the majority class, the best cross-validation F1 score can still be very low. This is a modelling issue rather than a software error and should be addressed through imbalance-aware techniques.

5. Evaluation Metrics & Results

Metric	Meaning	Why it Matters
Accuracy	Correct predictions / all predictions	Useful overall, but can be misleading for imbalanced data
Precision	TP / (TP + FP)	How reliable a fraud alert is
Recall	TP / (TP + FN)	How much actual fraud is detected
F1 Score	Harmonic mean of precision and recall	Balances fraud detection and false alarms
Confusion Matrix	Counts of TN, FP, FN and TP	Shows exactly which mistakes the model makes

Table 6: Evaluation Metrics

A deterministic reference run of the submitted preprocessing pipeline was used to sanity-check model behaviour. The results below are intended as a project execution reference. The Logistic Regression and Naive Bayes scripts, when run with their submitted grid search, also selected configurations that predicted only the majority class on the test split. This is why accuracy alone must not be used to claim that the fraud detector is successful.

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.7485	0.0000	0.0000	0.0000
Gaussian Naive Bayes	0.7485	0.0000	0.0000	0.0000
K-Nearest Neighbors	0.6999	0.2647	0.1088	0.1542
Decision Tree	0.7345	0.2888	0.0383	0.0677
Random Forest	0.7485	0.0000	0.0000	0.0000

Table 7: Reference Model Results

The most useful lesson from these results is that the current feature representation and class distribution encourage several models to choose the safer majority-class prediction. KNN detects some fraudulent examples and therefore produces a non-zero F1 score, but recall remains low. The project should therefore be evaluated not only as a final classifier, but also as a complete experimental pipeline that exposes the next modelling steps clearly.

6. Technology Stack

Component	Library / Tool	Purpose
Programming Language	Python	Data processing and machine learning
Data Handling	pandas	CSV loading, statistics and feature creation
Machine Learning	scikit-learn	Pipelines, preprocessing, PCA, classifiers and metrics
Visualization	Matplotlib	Charts and confusion matrices
EDA Visualization	Seaborn	Countplots, boxplots, scatter plots and heatmaps
Data Format	CSV	Input dataset and suspicious-account output
Reproducibility	random_state=42	Consistent data split and tree-based results

Table 8: Technology Stack

The project is organized into separate Python files. preprocessing.py contains the reusable data-preparation function. eda.py and preprocessing_eda.py focus on exploratory analysis. Each machine learning algorithm has its own script, making it easy to execute and compare models independently. test_dataset.py provides a simple dataset sanity check before the heavier experiments are run.

7. Code Implementation

7.1 Core Preprocessing Pipeline

```
def load_and_prepare_data():
    df = pd.read_csv(csv_path)
    timestamp = pd.to_datetime(df["timestamp"], errors="coerce")
    df["hour"] = timestamp.dt.hour
    df["day"] = timestamp.dt.day
    df["month"] = timestamp.dt.month
    df["day_of_week"] = timestamp.dt.dayofweek
    df["referrer_length"] =
df["referrer_url"].fillna("").astype(str).str.len()
    df["referrer_https"] =
df["referrer_url"].fillna("").astype(str).str.startswith("https").astype(int)
    df["page_url_length"] = df["page_url"].fillna("").astype(str).str.len()
    df["page_https"] =
df["page_url"].fillna("").astype(str).str.startswith("https").astype(int)
    df.drop(columns=["click_id", "user_id", "ip_address", "timestamp",
                     "referrer_url", "page_url"], inplace=True)
    X = df.drop(columns=["is_fraudulent"])
    y = pd.to_numeric(df["is_fraudulent"])
```

This block summarizes the feature-engineering idea used by the submitted preprocessing.py file. Raw identifier fields are intentionally removed so that the model does not simply memorize unique IDs or addresses.

7.2 Numerical and Categorical Transformation

```
numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
])
preprocessor = ColumnTransformer(transformers=[
    ("numeric", numeric_pipeline, numeric_columns),
    ("categorical", categorical_pipeline, categorical_columns)
])
```

Using ColumnTransformer allows both feature types to be handled in one reusable object. OneHotEncoder with handle_unknown="ignore" also prevents prediction-time errors when an unseen category appears in the test data.

7.3 Common Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

7.4 Logistic Regression with Grid Search

```
pipeline = Pipeline(steps=[
    ("preprocessing", preprocessor),
    ("pca", PCA(n_components=0.95)),
    ("classifier", LogisticRegression(max_iter=2000))
])
param_grid = {
    "pca__n_components": [0.90, 0.95],
    "classifier__C": [0.1, 1, 10]
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
grid = GridSearchCV(pipeline, param_grid, cv=cv, scoring="f1", n_jobs=-1)
grid.fit(X_train, y_train)
y_pred = grid.best_estimator_.predict(X_test)
```

7.5 Evaluation

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
cm = confusion_matrix(y_test, y_pred)
print(classification_report(y_test, y_pred, zero_division=0))
```

7.6 Other Classifier Definitions

```
# Gaussian Naive Bayes
GaussianNB()
# tuned: var_smoothing = [1e-9, 1e-8, 1e-7]

# K-Nearest Neighbors
KNeighborsClassifier()
# tuned: n_neighbors = [3, 5, 7], weights = ["uniform", "distance"]

# Decision Tree
DecisionTreeClassifier(random_state=42)
# tuned: criterion = ["gini", "entropy"], max_depth = [5, 10, 20]

# Random Forest
RandomForestClassifier(random_state=42, n_jobs=-1)
# tuned: n_estimators = [100, 200], max_depth = [10, 20]
```

The separate model files repeat the same workflow intentionally: load prepared data, create a pipeline, define the search space, run stratified cross-validation, predict the test set, calculate classification metrics and display a confusion matrix. This repetition makes each experiment easy to run independently.

7.7 Exploratory Data Analysis Code

```
df = pd.read_csv(csv_path)
print(df.shape)
print(df.columns.tolist())
print(df.describe(include="all"))
print(df.isnull().sum())
print(df.duplicated().sum())
print(df["is_fraudulent"].value_counts())

sns.countplot(data=df, x="is_fraudulent")
sns.boxplot(data=df, x="is_fraudulent", y="click_duration")
sns.scatterplot(data=df, x="click_duration",
                y="bot_likelihood_score", hue="is_fraudulent")
sns.heatmap(df.select_dtypes(include=["int64","float64"]).corr(),
            annot=True, cmap="coolwarm", fmt=".2f")
```

The EDA code is not only for presentation. It helps verify that the dataset has been loaded correctly and provides early evidence about imbalance, scale differences and potentially useful relationships before model training begins.

8. Visualization and Discussion

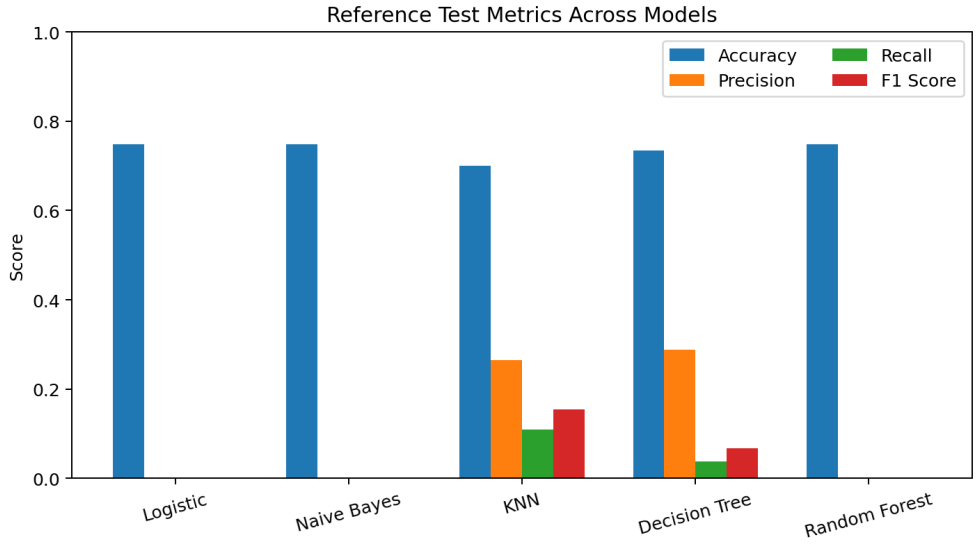

Figure 4: Model Evaluation Comparison

The comparison chart makes the class-imbalance issue visible. Accuracy remains close to the majority-class baseline for several classifiers, while their precision, recall and F1 values collapse. KNN gives the strongest F1 among the representative test runs shown here, but the recall indicates that most fraudulent clicks are still missed.

8.1 Interpretation of Confusion Matrices

For a fraud detector, the lower-right cell of a confusion matrix represents correctly detected fraudulent clicks (true positives), while the lower-left cell represents fraudulent clicks incorrectly accepted as genuine (false negatives). A model that predicts every sample as non-fraudulent can still appear strong by accuracy because the dataset contains many more class-0 records. Its confusion matrix, however, immediately reveals zero true-positive fraud detections.

8.2 Why Some Models Collapse to the Majority Class

The observed behaviour is consistent with an imbalanced dataset where the available transformed features do not create a sufficiently strong boundary for the minority class at the default decision settings. Logistic Regression and Naive Bayes can minimize overall classification error by favoring class 0. Random Forest can show similar behaviour when the positive class is not weighted and the tree ensemble finds only weak minority patterns.

8.3 What the Current Project Does Well

- Uses one reusable preprocessing function instead of duplicating data cleaning logic in each model file.
- Prevents category mismatch errors through OneHotEncoder(handle_unknown="ignore").
- Uses stratification so train and test sets retain the original class ratio.
- Uses F1 as the hyperparameter search score, which is more appropriate than accuracy for fraud detection.
- Compares multiple algorithm families rather than relying on a single classifier.

- Reports precision, recall, F1, classification report and confusion matrix, giving a fuller picture of model behaviour.

8.4 Recommended Improvements

- Add class_weight="balanced" for Logistic Regression, Decision Tree and Random Forest, then compare recall and F1 with the current baseline.
- Tune the probability decision threshold instead of always using the default 0.50 threshold.
- Try resampling methods such as random oversampling or SMOTE only inside cross-validation folds to avoid leakage.
- Use PR-AUC (average precision) and ROC-AUC in addition to point metrics.
- Investigate feature importance and permutation importance to understand which behavioural signals contribute to fraud decisions.
- Evaluate gradient-boosting classifiers such as XGBoost, LightGBM or HistGradientBoosting when allowed by the project scope.
- Consider removing PCA for tree-based models; trees may benefit from the original one-hot feature structure and do not require scaling in the same way as distance-based methods.
- Validate on a separate time period or unseen traffic source to measure real-world generalization.

These improvements do not invalidate the present work. They are the natural next stage after establishing a clean experimental pipeline and observing where the baseline models fail.

8.5 Practical Use of the Model

In a practical advertising system, the classifier could be placed between click logging and billing or analytics. Each incoming click would be converted into the same engineered features used during training. The model would then return a fraud probability or class. High-risk clicks could be sent for review, excluded from billing, rate-limited, or combined with rule-based checks.

For such deployment, probability calibration, threshold selection and monitoring are essential. A model should not be judged only by one offline test split. Fraud patterns change over time, so performance should be checked regularly and the model retrained when the traffic distribution changes.

8.6 Ethical and Operational Considerations

Fraud detection can affect users and advertisers, so false positives need careful handling. The system should avoid using personally identifying values as direct memorization features, log the reason for automated actions where possible, and provide a review path for high-impact decisions. The preprocessing step already moves in this direction by dropping click_id, user_id and ip_address before training.

9. Conclusion and Future Scope

This project presents a complete machine learning workflow for classifying click events as fraudulent or non-fraudulent. The dataset contains 55,000 records and combines behavioural, device, network and URL-related information. The implementation performs feature engineering, mixed-type preprocessing, dimensionality reduction, stratified splitting, cross-validated hyperparameter search and multi-metric evaluation.

A major finding of the experiments is that overall accuracy can be misleading in this problem. Because 74.86% of the records are non-fraudulent, a model can obtain an accuracy close to 0.75 while detecting no fraud at all. The reference executions show this behaviour in several baseline models. KNN identifies a limited number of fraudulent records, but the recall is still too low for a production-quality detector.

The value of the project is therefore twofold. First, it provides a structured and reusable implementation covering EDA, preprocessing and five common classification algorithms. Second, it clearly reveals the modelling challenge that must be solved next: improving minority-class detection without creating an excessive number of false alarms.

9.1 Future Scope

- Use class weights and threshold tuning to directly optimize fraud recall and F1 score.
- Compare oversampling and cost-sensitive learning inside cross-validation.
- Add PR-AUC, ROC-AUC and calibration curves to the evaluation chapter.
- Build an API or web interface that accepts click features and returns a fraud risk score.
- Store flagged cases in a review table and track analyst feedback for later retraining.
- Monitor model drift as user behaviour, bots and advertising traffic patterns change.

With these improvements, the project can evolve from a strong academic experimentation pipeline into a more practical fraud-detection system.

REFERENCES / SOFTWARE DOCUMENTATION

1. Scikit-learn documentation: preprocessing, Pipeline, ColumnTransformer, PCA, GridSearchCV and classification metrics.

2. pandas documentation: CSV loading, DataFrame operations and descriptive statistics.

3. Matplotlib documentation: data visualization and plotting.

4. Seaborn documentation: statistical plotting used in the exploratory analysis.

5. Project source files supplied with this report: preprocessing.py, eda.py, preprocessing_eda.py, logistic_regression.py, naive_bayes.py, knn.py, decision_tree.py, random_forest.py and test_dataset.py.

Note: The explanatory text in this report has been written specifically for this project and is not copied from the reference report. Standard algorithm names, library names and mathematical metric terminology are necessarily conventional.